

Deep Learning Based CT Reconstruction and Enhancement

Technical Communication Project Report

Hrushikesh Waghmare (23324025) Pushkar Kumar (23411026)

April 26, 2026

Abstract

This report is, about the status of a project that uses computed tomography to rebuild images. The project uses a computer program that takes in data from X-ray machines and other information from the scanner. It then rebuilds pictures using a special method called cone-beam filtered backprojection. After that it uses learning to improve the pictures.

The computer program is set up in a way that makes it easy to use and get the results every time. The report talks about how the data moves through the program how it understands the geometry of the scanner and how it rebuilds the images. It also talks about how the program creates training datasets and what the limitations of the program're. The computed tomography reconstruction project is still being worked on. This report shows what has been done so far with the computed tomography reconstruction project.

1 Introduction

The computed tomography scanner takes X-ray pictures of an object from different angles and then uses those pictures to figure out what is inside the object. When everything is working perfectly the scanner takes a lot of pictures from around the object, which makes it easy to see what is inside.. In real life things are not always perfect. The pictures can be blurry the scanner may not be able to take pictures from around the object and it may have to take fewer pictures than it wants to because it takes too long or uses too much radiation. When that happens the picture of what's inside the object can be blurry or have streaks in it and it can be hard to see the small details.

The goal of this project is to find a way to deal with these problems in a way. The project uses a combination of methods that are based on physics and new methods that use computers to learn and improve. The old methods give us a picture of what's inside the object that is easy to understand and look at. Then the computer uses that picture to learn how to either take the X-ray pictures and turn them directly into a picture of what's, inside the object or to take a blurry picture and make it clearer. The computed tomography scanner and the project are focused on improving the tomography images.

The current code supports four main modes:

1. **Classical:** 3D cone-beam FDK reconstruction using ASTRA.
2. **Sinogram:** sparse-view sinogram to image reconstruction.
3. **Enhance:** full-sinogram to FDK-style image enhancement.
4. **U-Net:** post-processing on degraded FDK reconstructions.

2 Project Layout

The repository is set up in a way that keeps things simple. Data handling is one thing geometry parsing is another. Then there is reconstruction and model training. These things are all separate, from each other. The repository is organized so that data handling, geometry parsing, reconstruction and model training are separated cleanly which means the repository is organized in a way that data handling and geometry parsing and reconstruction and model training're all separate.

- `src/ct_recon/`: core library code for loading scans, parsing geometry, simulating degradation, building training pairs, and defining models.
- `scripts/`: command-line entry points for the different workflows.
- `scripts/run_pipeline.py`: master dispatcher that launches the selected mode.
- `data/sample_1` and `data/sample_2`: sample-specific projection folders and scanner metadata.
- `outputs/`: generated reconstructions, datasets, previews, and checkpoints.
- `docs/reports/`: LaTeX sources for the report.

The main command used throughout the project is:

```
python scripts/run_pipeline.py {classical,sinogram,enhance,unet}
```

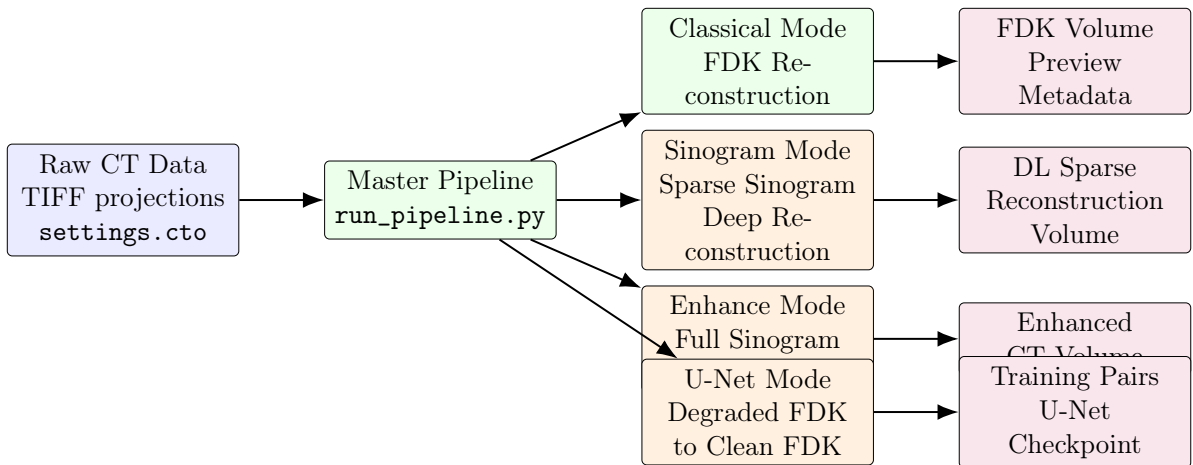


Figure 1: High-level view of the pipeline. The same raw CT input can be routed through a classical or learning-based workflow.

3 Data and Geometry

Each sample needs two things: a directory of TIFF projections and a settings.cto file that stores scanner metadata. The loader reads the projection stack into a 3D array with shape

(number of projections, detector rows, detector columns).

The repository is set up to keep things. Data handling is one thing. Geometry parsing is another thing. Then there is reconstruction and model training. These things, like data handling and geometry parsing and reconstruction and model training are all separate, from each other.

The repository is organized so that data handling and geometry parsing and reconstruction and model training are separated cleanly. This means that the repository is organized in a way that keeps data handling and geometry parsing and reconstruction and model training all separate.

That format is used consistently across reconstruction scripts and training scripts.

The geometry parser takes the scanner settings. Turns them into something that is organized and easy to understand. The geometry parser looks at things like the projection count, the range, the size of the detector the space between the detector pixels, the distance from the source to the object the distance from the source, to the detector, the center of rotation the vertical center and the usable axial range of the scanner. These things are what the geometry parser needs to create the cone-beam geometry that ASTRA is expecting from the scanner.

The geometry parser generates projection angles by going through the scanners range:

$$\theta_i = \theta_0 + i\Delta\theta, \quad \Delta\theta = \frac{\text{angle range}}{N}.$$

```
projections = int(scan["projections"])
angle_range_deg = float(scan["angle range"])
direction = int(recon["direction"])

start_angle_deg = 0.0
stop_angle_deg = start_angle_deg + direction * angle_range_deg

angles_deg = np.linspace(
    start_angle_deg,
    stop_angle_deg,
    projections,
    endpoint=False,
    dtype=np.float32,
)

angles_rad = np.deg2rad(angles_deg).astype(np.float32)
```

Listing 1: Building the projection angle array from scanner metadata

4 Classical Reconstruction

The classical path is the most important reference point in the project. It is used both as a baseline for visual comparison and as a source of target images for the learning-based pipelines.

4.1 Slice-wise FBP Check

An early test in the repository uses a 2D reconstruction method. This method is called filtered backprojection.

* For one row of detector data we create a sinogram by stacking the row from all angles. The sinogram is then. Backprojected across the image.

That method is not the way we will process data but it helps us debug and check that the projection data is being read correctly.

We use this method to make sure everything is working as expected with the projection data.

It is a step, for checking the data.

$$S(s, \theta) \rightarrow \tilde{S}(s, \theta) \rightarrow f(x, y)$$

4.2 FDK with ASTRA

The main way to rebuild the image uses ASTRA toolbox and its FDK implementation that works with CUDA.

The steps are easy to follow:

1. First I load all the projection images.
2. If needed I make the detector images smaller.
3. Next I change the intensity measurements to attenuation values.
4. Then I create the geometry, for the cone-beam projection.
5. After that I build the geometry for the reconstruction volume.
6. I run the FDK_{CUDA} .
6. Finally I trim the range and save the volume.

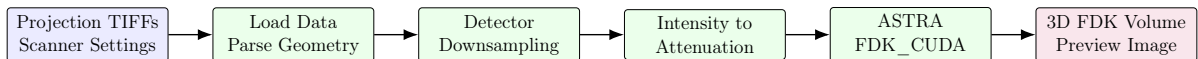


Figure 2: Classical reconstruction mode. The projection stack is converted to attenuation data and reconstructed as a 3D CT volume.

Detector downsampling uses block averaging so that the signal is reduced without simply discarding pixels:

$$P_{\text{ds}}(i, j) = \frac{1}{k^2} \sum_{u=0}^{k-1} \sum_{v=0}^{k-1} P(ki + u, kj + v).$$

Before reconstruction, the projections are converted from raw intensity to line-integral style attenuation values using a stable version of the Beer-Lambert transform:

$$\mu L = -\log \left(\frac{I}{I_0} \right).$$

```
def convert_to_attenuation(projections, air_percentile=99.9):
    projections = projections.astype(np.float32, copy=False)

    air_level = np.percentile(
        projections,
        air_percentile,
        axis=(1, 2),
        keepdims=True,
    )

    air_level = np.maximum(air_level, 1.0)
    normalized = np.clip(projections / air_level, 1e-6, 1.0)
    attenuation = -np.log(normalized)
    return attenuation.astype(np.float32, copy=False)
```

Listing 2: Intensity-to-attenuation conversion

5 Degradation and Training Data

The parts of the project that are, about learning need two kinds of things: inputs and perfect targets. The repository makes these inputs in two different ways: it makes them in projection space. It makes them in image space.

The kinds of degradation that the repository supportsre

- picking only a few views
- picking only a few angles
- adding Gaussian noise
- adding Poisson noise.

The Sparse-view CT technique only keeps every projections, which is the k -th one and that helps to reduce the cost of the scan.. This method also introduces some problems with the images because of angular undersampling. The Limited-angle CT technique is different it only keeps the projections from a part of the rotation, which means it is missing some important information from other directions. The people who made this system also added noise, to the simulations so that the Sparse-view CT and Limited-angle CT techniques can be tested in conditions that're more like real life.

```
def sparse_view_subset(projections, angles_rad, step):
    if step <= 0:
        raise ValueError("step must be positive")

    indices = np.arange(0, projections.shape[0], step, dtype=np.
        int32)
    return projections[indices], angles_rad[indices], indices
```

Listing 3: Selecting every kth projection for sparse-view CT

For the U- training path the project makes pairs of input and target data that match up. It does this by putting all the data then by putting together the data that is not complete. The project then cuts both sets of data to the size and after that it takes slices from the middle of the volumes. This way the project keeps things simple. The U-Net can focus on fixing the parts of the data that are missing. The U-Net training path is important because it helps the model learn how to restore the missing parts of the data which's what the U-Net is supposed to do.

```
def center_crop_to_match(source, target_shape):
    z, y, x = source.shape
    tz, ty, tx = target_shape

    z0 = (z - tz) // 2
    y0 = (y - ty) // 2
    x0 = (x - tx) // 2

    return source[z0:z0+tz, y0:y0+ty, x0:x0+tx]

def align_pair_shapes(input_volume, target_volume):
    shared_shape = tuple(
        min(a, b) for a, b in zip(input_volume.shape,
            target_volume.shape)
    )

    return (
        center_crop_to_match(input_volume, shared_shape),
        center_crop_to_match(target_volume, shared_shape),
    )
```

Listing 4: Aligning degraded and reference CT volumes

6 Learning-Based Pipelines

The repository has two ways to work with projections and one way to work with images. These methods are similar: they make a pair of items to learn from they use a kind of loss called L1 loss to train and they save the best result for later.

6.1 Sparse Sinogram Reconstruction

We have a workflow for when we do not have all the views we need. This sparse sinogram workflow takes a sinogram from a stack of attenuation keeps the views we are interested in makes the result the right size and then uses a special kind of network to predict what a CT slice should look like. This network is made up of an encoder and a decoder. The sparse sinogram workflow is very useful, for sinogram reconstruction.

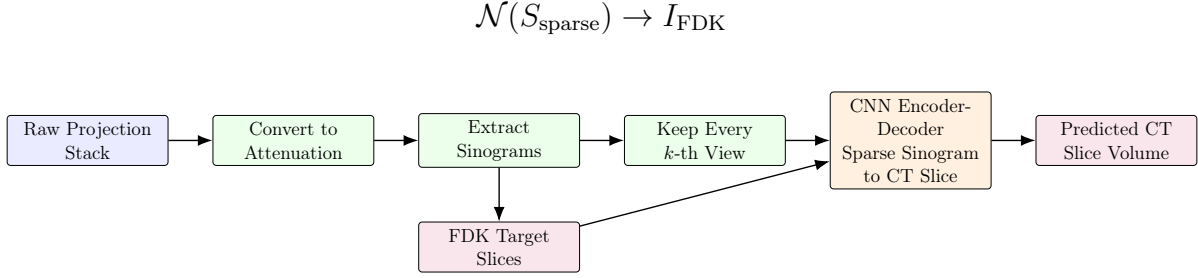


Figure 3: Sparse sinogram reconstruction. The model learns a direct mapping from undersampled projection data to image space.

6.2 Full Sinogram Enhancement

The enhance mode keeps the sinogram instead of a sparse one. This means it does not throw away much information as the sparse view setup. The enhance mode is also not as aggressive as the view setup. It is like a learning process that makes things look better rather than just solving a math problem. The target is still a reference FDK slice.. The input for the enhance mode has more of the original projection information, from the full sinogram.

$$\mathcal{N}(S_{\text{full}}) \rightarrow I_{\text{FDK}}$$

6.3 U-Net Enhancement

The U-Net workflow is really about making images look better especially when we are working with slices. It takes the not great FDK slices and uses them as input and then it looks at the good FDK slices as the target to work towards. The way it is set up is pretty standard it has an encoder and a decoder with some connections that help it keep the important parts of the image while making the details clearer. The U-Net workflow is about image-to-image enhancement, on these reconstructed slices.

$$\mathcal{N}(\hat{f}_d[z, :, :]) \rightarrow \hat{f}_{\text{ref}}[z, :, :]$$

7 Training Procedure

The training process is watched over in two areas: the projection part and the image part. The training process, for the model is pretty straightforward. We do a things: we move a group of items to the device that is available then we do a forward pass then we figure out the L1 loss then we go backwards to see what went wrong and we use Adam to make the model better. The code saves the model and the last model so that we can start training again or look at the models later to see which one is better.

```
model.train()

for inputs, targets in train_loader:
    inputs = inputs.to(device)
    targets = targets.to(device)

    predictions = model(inputs)
    loss = criterion(predictions, targets)

    optimizer.zero_grad(set_to_none=True)
    loss.backward()
    optimizer.step()
```

Listing 5: Core supervised training step

The main loss is L1:

$$\mathcal{L}_{L1} = \frac{1}{N} \sum_{i=1}^N |\hat{y}_i - y_i|.$$

Validation uses PSNR on normalized images:

$$\text{PSNR} = 10 \log_{10} \left(\frac{1}{\text{MSE}} \right).$$

8 Master Pipeline and Outputs

The file `scripts/run_pipeline.py` is where you start with this repository. It picks the mode you want. Sends you the

8.1 Classical Mode

```
python scripts/run_pipeline.py classical -sample sample_1
```

This mode runs only the classical reconstruction and saves the reconstructed volume, a preview image, and metadata.

8.2 Sinogram Mode

```
python scripts/run_pipeline.py sinogram -sample sample_1 -epochs 50
```


This mode is used to build some training data that's not complete it trains a model that is used for sinogram and then it runs a process to create a volume that is made from the data it has. The main thing this mode does is build this training data. Use the sinogram model to make a reconstructed volume.

8.3 Enhance Mode

```
python scripts/run_pipeline.py enhance -sample sample_1 -epochs 50
```

This mode is similar to sinogram mode, but it keeps the full sinogram as input.

8.4 U-Net Mode

```
python scripts/run_pipeline.py unet -sample sample_1 -epochs 50
```

This mode creates degraded/reference reconstruction pairs and trains the 2D U-Net on aligned slices.

The outputs are kept in folders that're specific to each sample like `<code>outputs/samplex_pipeline/ </code>` .*You will find things like volumes the datasets used for training, model checkpoints, directories for*

9 Current State, Limits, and Next Steps

The project already does everything from taking the projections to making the volumes look better. It gets the scanner information uses FDK to put the data makes it look like the conditions were not so good and teaches the computer to fix the projections and the images.

The problems we have now are because of how things are not because of the ideas:

- * We still have to get the U-Net to work fully with the pipeline.
- * We mostly look at how good it's by using L1 loss and PSNR.
- * The U-Net part only looks at one slice of the image at a time so it does not use all the information from the 3D image.
- * The files, with the projections and the full size reconstructions use a lot of memory.

The best next steps are:

1. create an U-Net script to make predictions
2. add reports, on SSIM and error maps
3. put classical and learned methods together for a comparison
4. improve tracking of experiments
5. expand learning models to 3D reconstruction.

10 Conclusion

This project shows how to combine methods for CT reconstruction.

It begins with data from a CT scanner and uses a traditional method, called FDK to create an initial image.

This image is then improved using machine learning techniques.

The result is a system that can be used for many things like

* creating a basic reconstruction * testing with limited data * enhancing image quality * trying out new models. The best part of this project is that it combines two approaches: one based on physics and the other on machine learning. This makes the system easy to understand and fix. It also makes it more useful for work on CT reconstruction. The project keeps the physics part. The machine learning part together in one workflow. That is what makes it easy to inspect and debug. The system is more useful as a basis for CT reconstruction work because of this. It is a starting point, for more advanced CT reconstruction projects. The combination of physics and machine learning makes it a unique and valuable tool.

References

- [1] W. van Aarle et al., “The ASTRA Toolbox: A platform for advanced algorithm development in electron tomography,” *Ultramicroscopy*, 2016.
- [2] O. Ronneberger, P. Fischer, and T. Brox, “U-Net: Convolutional Networks for Biomedical Image Segmentation,” *MICCAI*, 2015.
- [3] A. Paszke et al., “PyTorch: An Imperative Style, High-Performance Deep Learning Library,” *NeurIPS*, 2019.

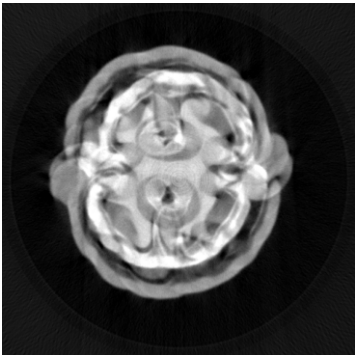
Sample 1 Preview Images

Added appendix page showing representative input and reconstruction outputs for sample 1.

Raw projection frame

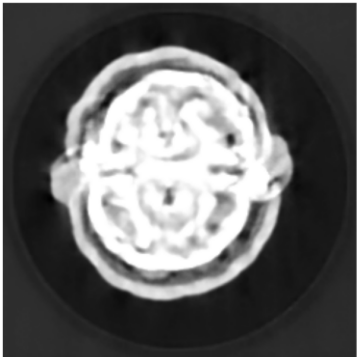


Classical FDK reconstruction (axial slice from volume)

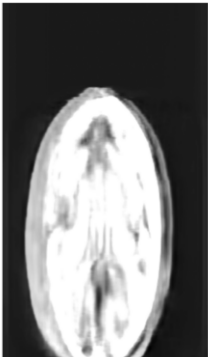


Sparse sinogram reconstruction preview

Axial z=220 (window: 0.013367759063839912 ± 0.04622746631503105)



Coronal y=128



Sagittal x=128

